36. Containers, Namespaces, and Networking

36. Containers, Namespaces, and Networking

Phase: Phase 11 — Containers and cloud Linux
Estimated time: 2 days
Slug: `36-docker-linux-net`

Learning objectives

- Name major namespaces (pid, net, mnt, user)
- Explain bridge networking basics
- Debug DNS inside containers
- Use host networking knowingly

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

A container is not a small VM. It is an ordinary Linux process placed in **namespaces** (what it can see) and **cgroups** (how much it can use). When a container cannot reach the internet, or two containers cannot talk, the answer is almost always in Linux networking you already know: network namespaces, veth pairs, bridges, iptables/nftables NAT, and routing. Understanding this turns “Docker networking is magic” into commands you can run.

Core concepts

Namespace	Isolates
pid	Process IDs (container sees itself as PID 1)
net	Interfaces, routes, ports, firewall rules
mnt	Mount table / root filesystem
uts	Hostname
ipc	Shared memory and queues
user	UID/GID mapping
cgroup	View of cgroup tree

Docker network drivers:

Driver	Behavior
bridge (default)	Containers on a private Linux bridge `docker0`, NAT to the outside
host	Shares the host's network namespace; no isolation, no port mapping
none	Only loopback
user-defined bridge	Like bridge, with DNS names between containers

The default Docker network is **bridge**.

```
container A (netns)        container B (netns)
  eth0 172.17.0.2            eth0 172.17.0.3
    |  veth                     |  veth
----+---------------------------+----  docker0 bridge 172.17.0.1
                |
             iptables NAT (MASQUERADE)  <-- outbound
             DNAT host:8080 -> 172.17.0.2:80  <-- published ports
                |
              host eth0 --> internet
```

Each container's `eth0` is one end of a **veth pair**; the other end is attached to the bridge on the host. Outbound traffic is masqueraded to the host IP; inbound published ports use DNAT rules.

Worked examples

List networks and inspect the default:

```
docker network ls
docker network inspect bridge --format '{{range .IPAM.Config}}{{.Subnet}}
{{.Gateway}}{{end}}'
ip addr show docker0
bridge link
```

See a container's namespace from the host:

```
docker run -d --name n1 alpine sleep 600
pid=$(docker inspect -f '{{.State.Pid}}' n1)
sudo ls -l /proc/$pid/ns
sudo nsenter -t "$pid" -n ip addr
sudo nsenter -t "$pid" -n ip route
```

Create a namespace by hand (no Docker) to see the mechanism:

```
sudo ip netns add lab
sudo ip link add veth-h type veth peer name veth-c
sudo ip link set veth-c netns lab
```

```
sudo ip addr add 10.99.0.1/24 dev veth-h && sudo ip link set veth-h up
sudo ip -n lab addr add 10.99.0.2/24 dev veth-c
sudo ip -n lab link set veth-c up && sudo ip -n lab link set lo up
ping -c1 10.99.0.2
sudo ip netns exec lab ping -c1 10.99.0.1
```

Cleanup: `sudo ip netns del lab; sudo ip link del veth-h 2>/dev/null`.

User-defined bridge with name resolution:

```
docker network create appnet
docker run -d --name db --network appnet redis:7
docker run --rm --network appnet alpine ping -c1 db
```

On the default bridge, names do not resolve; on a user-defined one they do.

Published ports and NAT rules:

```
docker run -d --name w -p 8080:80 nginx:alpine
sudo iptables -t nat -L DOCKER -n
curl -sI http://127.0.0.1:8080 | head -n 1
```

Host networking and none:

```
docker run --rm --network host alpine ip addr | head
docker run --rm --network none alpine ip addr
```

DNS inside containers:

```
docker run --rm alpine cat /etc/resolv.conf
docker run --rm --dns 1.1.1.1 alpine nslookup example.com
```

Mini-lab

1. Run two alpine containers on the default bridge. Ping by IP, then by name. Explain the difference.
2. Create `appnet`, repeat, and observe name resolution works.
3. Use `nsenter` to print the container's routes; identify the gateway as `docker0`'s address.
4. Publish a port, then find the DNAT rule in `iptables -t nat -S`.
5. Build the manual veth namespace lab and ping both directions. Clean up.
6. Name the default Docker network, which is the exercise answer.

Decision guide / failure modes

Symptom	Check
Container cannot reach internet	`ip_forward`, MASQUERADE rule, DNS in `resolv.conf`
Cannot reach container by name	use a user-defined network
Published port unreachable	firewall (ufw/firewalld), `ss -ltnp`, correct `-p`
Port conflict	another process owns host port
Two stacks clash	overlapping subnets; change `default-address-pools`

Failure modes:

- `sysctl net.ipv4.ip_forward=0` breaks container egress.
- ufw/firewalld rules vs Docker's own iptables rules: published ports can bypass ufw; test from outside.
- `--network host` removes isolation and ignores `-p`.
- Assuming `localhost` inside a container is the host; it is the container.
- Corporate VPN subnets overlapping `172.17.0.0/16` cause strange routing.

Common mistakes

- –network host as default habit
- Assuming localhost in container is the host
- Ignoring cgroup memory limits

Exercises

Work in `exercises/36-docker-linux-net/`. Read that folder's `README.md` fully before coding.

```
cd exercises/36-docker-linux-net
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `default_docker_network` in `exercise.sh` so it prints the name of the network driver and network that new containers join when you do not pass `--network`. Use the lowercase one-word name shown by `docker network ls`. If Docker is available, confirm with `docker network ls` and `docker inspect` on a throwaway container, and record the subnet of that network in your notes. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/36-docker-linux-net/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What do namespaces isolate?
2. Default docker network mode?
3. localhost confusion?

Further reading

- https://docs.docker.com/network/